COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	09/15/2026	**DATE SUBMITTED:**	10/03/2026
NAME:	Lance Cristopher S. Salcedo	**INSTRUCTOR:**	Engr. Jimlord Quejardo

SEATWORK 9.1 IMPLEMENTING TREES

1. PROCEDURE OUTPUT

binaryTree.h

```cpp
#ifndef BINARYTREE_H
#define BINARYTREE_H

template <typename T>
class BinaryTree{
    private:
        T key; //key
        BinaryTree<T>* leftChild; //pointer for left child mode
        BinaryTree<T>* rightChild; //pointer for right child mode

    public:
        BinaryTree(T rootObj); //constructor

        void insertLeft(T newNode); // insert left
        void insertRight(T newNode); // insert right

        BinaryTree<T>* getLeftChild(); //to access left child mode
        BinaryTree<T>* getRightChild(); //to access right child mode

        T getRootVal(); // get the root value node
        void setRootVal(T obj); //get the set root value

        bool isLeaf(); //node checker if its a leaf
};

//creates new root
template <typename T>
BinaryTree<T>::BinaryTree(T rootObj){
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
    std::cout << "Root was created\n";
```

```
33  }
34
35  template <typename T>
36  void BinaryTree<T>::insertLeft(T newNode){
37      //two types
38
39      //if there is no currently child
40      if (leftChild == nullptr){
41          leftChild = new BinaryTree<T>(newNode);
42      }
43      //if we want to insert a certain node
44      else{
45          //create a new node:
46          BinaryTree<T>* newChild = new BinaryTree<T>(newNode);
47          //move existing left child below the new node
48          newChild->leftChild = leftChild;
49          //make the new node the left child
50          leftChild = newChild;
51      }
52  }
53
54
55  template <typename T>
56  void BinaryTree<T>::insertRight(T newNode){
57      //if there is no currently child
58      if (rightChild == nullptr){
59          rightChild = new BinaryTree<T>(newNode);
60      }
61      else{
62          BinaryTree<T>* newChild = new BinaryTree<T>(newNode);
```

```
63              newChild->rightChild = rightChild;
64              rightChild = rightChild;
65          }
66      }
67
68      template <typename T>
69      BinaryTree<T>* BinaryTree<T>::getLeftChild(){
70          return leftChild;
71      }
72
73      template <typename T>
74      BinaryTree<T> *BinaryTree<T>::getRightChild(){
75          return rightChild;
76      }
77
78      template <typename T>
79      void BinaryTree<T>::setRootVal(T obj){
80          key = obj;
81      }
82
83      template <typename T>
84      T BinaryTree<T>::getRootVal(){
85          return key;
86      }
87
88      template <typename T>
89      bool BinaryTree<T>::isLeaf(){
90          return leftChild == nullptr && rightChild == nullptr;
91      }
92
93      #endif
```

main.cpp

```cpp
#include <iostream>
#include "binaryTree.h"

int main(){

    BinaryTree<char>* root = new BinaryTree<char>('A');
    root->insertLeft('B');
    root->insertRight('C');

    std::cout << "The root is: " << root->getRootVal();
    std::cout << "\nThe left child of root A is: " << root->getLeftChild()->getRootVal() << std::endl;
    std::cout << "\nThe right child of root A is: " << root->getRightChild()->getRootVal() << std::endl;

    root->getLeftChild()->insertLeft('D');
    root->getLeftChild()->insertRight('E');
    std::cout << "The left child of root B: " << root->getLeftChild()->getLeftChild()->getRootVal() << std::endl;
    std::cout << "The right child of root B: " << root->getLeftChild()->getRightChild()->getRootVal() << std::endl;

    root->getRightChild()->insertLeft('F');
    root->getRightChild()->insertRight('G');
    std::cout << "The left child of root C: " << root->getRightChild()->getLeftChild()->getRootVal() << std::endl;
    std::cout << "The right child of root C: " << root->getRightChild()->getRightChild()->getRootVal() << std::endl;

    return 0;
}
```

Output:

```
Root was created
The root is: A
The left child of root A is: B
The right child of root A is: C
Root was created
Root was created

The left child of root B: D
The right child of root B: E
Root was created
Root was created

The left child of root C: F
The right child of root C: G
```

Documentation:

Code Snippet	Function

<table>
<tr>
<td>

```
template <typename T>
class BinaryTree{

    private:
    //key
    T key;

    //pointers left child & right child
    BinaryTree<T>* leftChild;
    BinaryTree<T>* rightChild;

    public:

        //constructor
        BinaryTree(T rootObj);

        //insert left and right
        void insertLeft(T newNode);
        void insertRight(T newNode);
        //to access child nodes

        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();

        //get root value & set root node
        T getRootVal();
        void setRootVal(T obj);

        //node checker if leaf
        bool isLeaf();
};
```

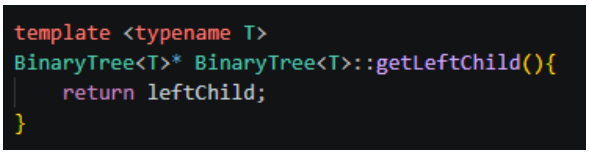
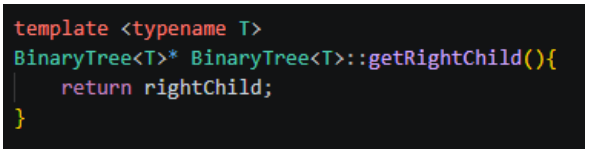
</td>
<td>This function creates the class for BinaryTree. In the private is where we put the key and the pointer of the left child and right child. In the public is where we put the constructor, where we insert both left and right, the access of both child nodes, getting the root value and root node as well as the node checker for leaves.</td>
</tr>
<tr>
<td>

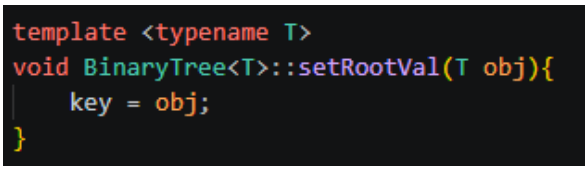
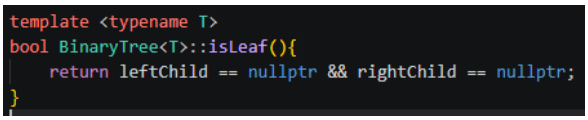
```
//creates new root
template <typename T>
BinaryTree<T>::BinaryTree(T rootObj){
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
}
```

</td>
<td>The function creates a new root for the binary tree.</td>
</tr>
<tr>
<td>

```
template <typename T>
T BinaryTree<T>::getRootVal(){
    return key;
}
```

</td>
<td>This function returns the key after getting the root value.</td>
</tr>
</table>

	This function is where we insert the left root in the new node. The function checks if there is no child, if it doesn't it becomes the new left child and also checks if the value is going to insert a certain node. It also moves for the existing left child below the new node and makes the new node the left child.
	This function is where we insert the right root in the new node. The function checks if there is no child, if it doesn't it becomes the new right child and also checks if the value is going to insert a certain node. It also moves for the existing right child below the new node and makes the new node the right child.
	This function returns the left child after getting the left child.
	This function return the right child after getting the right child.

	This function sets the root value as the key is now the obj.
	This function is to check if the root is a leaf if both the left child and right child are null pointers.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.